



PRACTICA 2

Estructura de Datos

Javier Crespo

Índice

Introducción	2
Metodología	3
Especificaciones TAD's	3
Interfaz	5
Implementación	6
Código Principal.....	9
Introducción	14

Introducción

Este código en Python recrea, de manera simulada, el proceso que ocurre en una biblioteca cuando los libros son devueltos y posteriormente clasificados según su género. La simulación se ejecuta en tiempo real, utilizando ciclos temporales para imitar la llegada de nuevos libros y su ordenación. Además, se incluye un mecanismo de notificación que indica si un libro se ha devuelto con retraso. La estructura general se basa en el uso de árboles binarios de búsqueda, lo que organiza los libros de forma eficiente, facilitar búsquedas rápidas y gestionar de forma dinámica la inserción y eliminación de elementos en función de su ISBN y otros atributos.



Metodología

Especificaciones TAD's

MÉTODO ALGEBRAICO o AXIOMÁTICO

NOMBRE

Arbol[l]

CONJUNTOS

A Conjunto de Arboles

L Conjunto de elementos

M Conjunto de mensajes {"La lista está vacía"}

B Conjunto de booleanos {true, false}

SINTAXIS

CrearArbol: $\rightarrow A$

Insertar: $L \times A \rightarrow A$

Buscar: $L \times A \rightarrow B$

Borrar: $L \times A \rightarrow A$

Recorrer_inorder $A \rightarrow M$

Recorrer_preorder $A \rightarrow M$

Recorrer_postorder $A \rightarrow M$

	CrearArbol	Insertar
Buscar	Buscar(CrearArbol)	Buscar(L,insertar(L,A))
Borrar	Borrar(CrearArbol)	Borrar(L,insertar(L,A))
Recorrer_inorder	Recorrer_inorder(CrearArbol)	Recorrer_inorder(insertar(L,CrearArbol))
Recorrer_preorder	Recorrer_preorder (CrearArbol)	Recorrer_preorder(insertar(L,CrearArbol))
Recorrer_postorder	Recorrer_postorder (CrearArbol)	Recorrer_postorder (insertar(L,CrearArbol))

SEMANTICA

Buscar(L, CrearArbol) = False	Buscar(L, insertar(L,A)) = True
Borrar(L, CrearArbol) = False	Borrar(L, insertar(L,A)) = True
Recorrer_inorder(CrearArbol) = M	Recorrer_inorder(insertar(L,CrearArbol)) = L
Recorrer_preorder (CrearArbol) = M	Recorrer_preorder(insertar(L,CrearArbol)) = L
Recorrer_postorder (CrearArbol) = M	Recorrer_postorder (insertar(L,CrearArbol)) = L
Insertar (L,CrearArbol)= A	

MÉTODO ALGEBRAICO o AXIOMÁTICO

NOMBRE

Libro[I]

CONJUNTOS

L Conjunto de Libros

I Conjunto de números enteros

S Conjunto de caracteres Sting

B Conjunto de booleanos {true, false}

M = Conjunto de mensajes

SINTAXIS

CrearLibro $\rightarrow$ L

Interfaz

```
# INTERFAZ
class LibroGenerico(ABC):
    @abstractmethod
    def __init__(self, ISBN, titulo, paginas, genero, retraso):
        pass

class ArbolGenerico(ABC):

    @abstractmethod
    def insertar(self):
        pass

    @abstractmethod
    def insertar_recursivo(self):
        pass

    #INORDER
    @abstractmethod
    def recorrer_inorder(self):
        pass

    @abstractmethod
    def recorrer_inorder_recursivo(self):
        pass
```

Defino dos clases abstractas utilizando la librería abc en Python, que sirve para crear interfaces o plantillas de clases que otras clases deben implementar:

class LibroGenerico(ABC):

Indica que LibroGenerico es una clase abstracta.

El método `__init__` está decorado con **@abstractmethod** y **pass**, lo que significa que no está implementado en esta clase y que cualquier subclase concreta de LibroGenerico debe proporcionar su propia implementación de este método.

En el contexto de una biblioteca de libros, LibroGenerico define la estructura básica de un libro (parámetros como Siguiente, ISBN, titulo, paginas, genero, retraso), pero deja la implementación detallada a sus subclases.

class ArbolGenerico (ABC):

Esta es una clase **abstracta**, diseñada como la interfaz de una estructura de datos de tipo árbol binario de búsqueda. Todos los métodos incluidos en esta clase están decorados con **@abstractmethod**, lo que obliga a cualquier clase que herede de **ArbolGenerico** a implementar estos métodos. Esto garantiza que todas las implementaciones concretas de árboles binarios de búsqueda proporcionen una interfaz consistente para operaciones clave, como la inserción, los recorridos (inorden, preorden, postorden), la búsqueda y la eliminación de nodos, facilitando así su uso y mantenimiento en aplicaciones más complejas.

Implementación

```
# IMPLEMENTACIÓN
class Libro(LibroGenerico):
    def __init__(self):
        generos = ["ensayo", "novela", "poesía"]
        self.derecha = None
        self.izquierda = None
        self.ISBN = random.randint(100000, 999999) # 6 dígitos
        longitud_titulo = random.randint(10, 20)
        self.titulo = ''.join(random.choices(string.ascii_letters, k=longitud_titulo))
        self.genero = random.choice(generos)
        self.paginas = random.randint(10, 99) # 2 dígitos
        self.retraso = random.choice([True, False])

class Arbol(ArbolGenerico):
    def __init__(self):
        self.raiz=None

    def insertar(self,Libro):
        self.raiz = self.insertar_recursivo(self.raiz,Libro)

    def insertar_recursivo(self,actual,Libro):
        if actual is None:
            return Libro
        else:
            if(Libro.ISBN > actual.ISBN):
                actual.derecha = self.insertar_recursivo(actual.derecha,Libro)
            elif(Libro.ISBN < actual.ISBN):
                actual.izquierda = self.insertar_recursivo(actual.izquierda,Libro)
            return actual

    #INIRDER
    def recorrer_inorder(self):
        if self.raiz is None:
            print("El Árbol está vacío.")
            return None

        resultado = []
        self.recorrer_inorder_recursivo(self.raiz, resultado)
        print(", ".join(str(isbn) for isbn in resultado))

    def recorrer_inorder_recursivo(self, libro_actual, resultado):
        if libro_actual:
            self.recorrer_inorder_recursivo(libro_actual.izquierda, resultado)
            resultado.append(libro_actual.ISBN)
```

Clase Libro

La clase Libro es una implementación concreta de la clase abstracta LibroGenerico.

- **Siguiente:** se inicializa en None, pues servirá para enlazar con el siguiente libro en la pila.
- **ISBN:** se genera un número entero de 6 dígitos de forma aleatoria.
- **título:** se crea una cadena aleatoria de 10 a 20 caracteres alfabéticos.
- **género:** se elige aleatoriamente entre “ensayo”, “novela” o “poesía”.
- **páginas:** se genera un entero aleatorio de 2 dígitos (entre 10 y 99).
- **retraso:** booleano aleatorio que puede ser True o False.

Método mostrar: Imprime en pantalla los datos del libro (uno por línea): ISBN, título, género, número de páginas y si tiene retraso o no.

Clase Arbol

Arbol es una implementación concreta de la clase abstracta `ArbolGenerico`, que gestiona un árbol binario de búsqueda para almacenar libros. Su diseño recurre a métodos recursivos para garantizar la correcta ordenación y manipulación de los nodos (libros). A continuación se detalla el funcionamiento de sus operaciones clave:

En la inicialización del Árbol se declara el atributo `self.raiz` y se asigna el valor `None`.

Insertar:

Inserta un nuevo libro en el árbol binario de búsqueda de forma recursiva:

1. Se invoca el método *insertar*, que a su vez llama al método recursivo *insertar_recursivo* pasándole la raíz actual y el libro a insertar.
2. **Caso base:**
 - Si el nodo actual es **None**, se asigna el libro a esa posición, convirtiéndolo en un nuevo nodo.
3. **Comparación de ISBN:**
 - Si el ISBN del libro a insertar es mayor que el del nodo actual, se recorre el subárbol derecho.
 - Si es menor, se recorre el subárbol izquierdo.
4. Así se garantiza que, al final, el libro se inserte en la posición correcta, manteniendo el orden del árbol.

Recorrer:

La clase `Arbol` dispone de métodos para recorrer y mostrar los nodos en distintos órdenes (inorden, preorden y postorden). Por ejemplo, el recorrido inorden se realiza de la siguiente manera:

1. Si el árbol está vacío, imprime "El Árbol está vacío."
2. Si no está vacío, se inicia una lista para almacenar los ISBN de los nodos e imprimirlos posteriormente.
3. Se recorre de forma recursiva el subárbol izquierdo, se añade el ISBN del nodo actual a la lista, y después se recorre el subárbol derecho.
 - Para los preorden antes de recorrer el de forma recursiva el subárbol izquierdo se añade a la lista de nodos y para los postorden se añade a la lista de nodos al final de recorrer el subárbol derecho.

Borrar:

Elimina un libro en función de su ISBN siguiendo estos pasos:

1. Se llama al método *borrar*, que invoca el método recursivo *borrar_recursivo* con la raíz y el ISBN a eliminar.
2. **Caso base:**
 - Si el nodo actual es **None**, se retorna sin cambios, ya que el libro no se encuentra en esa rama.
3. **Recorrido para encontrar el nodo:**
 - Si el ISBN a borrar es mayor que el del nodo actual, se recorre el subárbol derecho.
 - Si es menor, se recorre el subárbol izquierdo.
4. **Una vez encontrado el nodo:**
 - **Caso 1 (Nodo sin hijos):**
 - Si el nodo es hoja (no tiene hijos), se elimina asignando **None** a esa posición.
 - **Caso 2 (Nodo con un solo hijo):**
 - Si el nodo tiene únicamente un hijo, se reemplaza por ese hijo (ya sea el izquierdo o el derecho).
 - **Caso 3 (Nodo con dos hijos):**
 - Se busca el nodo con el menor ISBN del subárbol derecho como sucesor del nodo a borrar.
 - Se copian los datos del sucesor en el nodo a eliminar y, a continuación, se elimina el sucesor del subárbol derecho de forma recursiva.

Buscar:

Realiza la búsqueda de un libro en el árbol basándose en su ISBN de la siguiente forma:

1. Se invoca el método *buscar*, que a su vez llama a su método recursivo *buscar_recursivo* empezando desde la raíz.
2. **Recorrido del árbol:**
 - Si el nodo actual es **None**, se termina la búsqueda, ya que el libro no se encuentra en esa rama.
 - Si el ISBN buscado es menor que el del nodo actual, se continúa la búsqueda en el subárbol izquierdo; de lo contrario, en el subárbol derecho.
3. Devuelve **True** si se encuentra el libro y **False** si no existe en el árbol.

Código Principal

```
def obtener_nodos(nodo):  
    return [] if nodo is None else [nodo] + obtener_nodos(nodo.izquierda) + obtener_nodos(nodo.derecha)
```

- **Función obtener_nodos:**

Esta función recopila todos los nodos de un árbol de forma recursiva y los devuelve en una única lista. Funciona de la siguiente manera:

1. **Caso base:**

- Si el parámetro nodo es None (es decir, se ha llegado al final de una rama), la función devuelve una lista vacía [].

2. **Caso recursivo:**

- Si nodo no es None, se crea una nueva lista que:
 - Contiene el nodo actual ([nodo]).
 - Se concatena con el resultado de llamar a obtener_nodos(nodo.izquierda), lo que recopila los nodos del subárbol izquierdo.
 - Se concatena además con el resultado de llamar a obtener_nodos(nodo.derecha), lo que recopila los nodos del subárbol derecho.

```
# Función para mostrar los Árboles  
def arboles(ABB1, ABB2, ABB3):  
    print("ABB1:", end="")  
    ABB1.recorrer_inorder()  
    print("\nABB2:", end="")  
    ABB2.recorrer_inorder()  
    print("\nABB3:", end="")  
    ABB3.recorrer_inorder()  
    print() # Salto de línea final
```

- **Función arboles:**

Esta función la creé para evitar la redundancia al imprimir los arboles

```
def simulacion(ABB1, ABB2, ABB3):

    tiempo_inicial = time.time()

    # Simulación durante 30 ciclos
    while time.time() - tiempo_inicial < 60:
        tiempo_actual = time.time()
        tiempo_transcurrido = int(tiempo_actual - tiempo_inicial)
        print(f"\nTiempo transcurrido: {tiempo_transcurrido} segundos")
        time.sleep(1)

        # Cada 5 segundos se simula la devolución de un libro
        if tiempo_transcurrido != 0 and tiempo_transcurrido % 5 == 0:
            print("-----")
            libro = Libro()
            if libro.genero == "ensayo":
                ABB1.insertar(libro)
                arboles(ABB1, ABB2, ABB3)
                print(f"Evento: Se devuelve el libro con id {libro.ISBN} al ABB1.")
            elif libro.genero == "novela":
                ABB2.insertar(libro)
                arboles(ABB1, ABB2, ABB3)
                print(f"Evento: Se devuelve el libro con id {libro.ISBN} al ABB2.")
            else:
                ABB3.insertar(libro)
                arboles(ABB1, ABB2, ABB3)
                print(f"Evento: Se devuelve el libro con id {libro.ISBN} al ABB3.")
            print("-----")

        # Cada 7 segundos se simula la ordenación de un libro
        if tiempo_transcurrido != 0 and tiempo_transcurrido % 7 == 0:
            print("-----")

            # Se escoge aleatoriamente uno de los tres árboles.
            arboles_con_libros = [arbol for arbol in [ABB1, ABB2, ABB3] if arbol.raiz is not None]
            arbol_seleccionado = random.choice(arboles_con_libros)

            # Se obtiene la lista de nodos (libros) en el árbol seleccionado
            nodos = obtener_nodos(arbol_seleccionado.raiz)

            if nodos:
                libro_seleccionado = random.choice(nodos)

                if libro_seleccionado.retraso:
```

- **Función simulación:**

Esta función simula el proceso de devolución y ordenación de libros en una biblioteca durante un período de tiempo definido.

1. **Inicialización del Tiempo:**

- Se registra el tiempo inicial (tiempo_inicial) para controlar la duración de la simulación.

2. **Bucle de Simulación:**

- La simulación se ejecuta dentro de un bucle while que continúa hasta que hayan transcurrido 60 segundos desde el inicio.
- En cada iteración, se calcula el tiempo transcurrido (tiempo_transcurrido) y se muestra en la consola.
- Se introduce una pausa de 1 segundo (time.sleep(1)) para simular el paso del tiempo.

3. **Simulación de Devolución de Libros (cada 5 segundos):**

- Cada 5 segundos, se crea un nuevo libro (Libro()) y se añade al árbol correspondiente según su género.
- Se muestra un mensaje indicando que un libro ha sido devuelto, junto con su ISBN.
- Se muestra el estado actual de los árboles .

4. **Se detecta retraso de un libro (cada 7 segundos):**

- Cada 7 segundos, se elige un libro aleatorio en un árbol aleatorio.
- Si el libro elegido tiene retraso se elimina
- Se muestra un mensaje indicando si el libro seleccionado fue eliminado junto con el estado de los árboles.

```

# MAIN
ABB1 = Arbol()
ABB2 = Arbol()
ABB3 = Arbol()

while True:
    print("\nMenú Principal")
    print("1. Insertar un libro en un ABB")
    print("2. Buscar cualquier libro en un ABB")
    print("3. Imprimir lista de libros en cualquier ABB")
    print("4. Borrar un libro introducido por teclado")
    print("5. Introducir un número y crear dicha cantidad de libros")
    print("6. Introducir un número y generar al azar dicha cantidad de libros, que se borrarán, si existen")
    print("7. Iniciar la simulación")
    print("8. Salir de la aplicación")

    opcion = input("Elige una opción: ")

    match opcion:
        case "1":
            libro = Libro()

            while True:
                numero = input("\nIntroduce un el ISBN (número de 6 dígitos): ")
                try:
                    numero = int(numero)
                    if 100000 <= numero <= 999999:
                        libro.ISBN = numero
                        break
                    else:
                        print("El ISBN debe tener exactamente 6 dígitos.")
                except ValueError:
                    print("Eso no es un número válido.")

            print("\n-----.")
            if libro.genero == "ensayo":
                ABB1.insertar(libro)
                arboles(ABB1, ABB2, ABB3)
                print(f"Evento: Se devuelve el libro con id {libro.ISBN} al ABB1.")
            elif libro.genero == "novela":
                ABB2.insertar(libro)
                arboles(ABB1, ABB2, ABB3)
                print(f"Evento: Se devuelve el libro con id {libro.ISBN} al ABB2.")
            else:
                ABB3.insertar(libro)
                arboles(ABB1, ABB2, ABB3)
                print(f"Evento: Se devuelve el libro con id {libro.ISBN} al ABB3.")
            print("\n-----.")

        case "2":
            while True:

```

Menú Interactivo:

El usuario puede elegir entre diferentes opciones, que incluyen:

- **Opción 1:** Insertar un libro en un ABB
 - Se crea un libro (con atributos aleatorios)
 - Se solicita al usuario el ISBN del libro
 - Se determina el género del libro y se inserta en el ABB correspondiente mediante insertar(libro)
 - Se muestra el estado actual de los ABB con arboles(ABB1, ABB2, ABB3)
 - **Fin**
- **Opción 2:** Buscar cualquier libro en un ABB
 - Se solicita al usuario el ISBN del libro
 - Se busca en los ABB utilizando buscar(ISBN) en cada uno
 - Se muestra un mensaje indicando en qué ABB se encontró el libro o si no fue hallado
 - **Fin**
- **Opción 3:** Imprimir lista de libros en un ABB
 - Se despliega un submenú para elegir el género (1: Ensayo, 2: Novela, 3: Poesía)
 - Se recorren y muestran los libros del ABB seleccionado mediante los métodos recorrer_inorder(), recorrer_preorder() y recorrer_postorder()
 - **Fin**

- **Opción 4:** Borrar un libro introducido por teclado
 - Se solicita el ISBN del libro a borrar
 - Se verifica en qué ABB se encuentra y se elimina con borrar(ISBN)
 - Se actualiza y muestra el estado del árbol con arboles(ABB1, ABB2, ABB3)
 - **Fin**
- **Opción 5:** Introducir un número y crear dicha cantidad de libros
 - Se solicita al usuario el número de libros a generar
 - Se crean e insertan los libros en sus respectivos ABB según su género
 - Se muestra el estado actualizado de los ABB
 - **Fin**
- **Opción 6:** Introducir un número y generar al azar dicha cantidad de libros, que se borrarán si existen
 - Se pide un número al usuario
 - Para cada libro generado, se verifica si ya existe en el ABB correspondiente:
 - Si existe, se elimina con borrar(ISBN)
 - Si no existe, no hace nada
 - Se muestra el árbol actualizado
 - **Fin**
- **Opción 7:** Iniciar la simulación
 - Se ejecuta la función simulacion(ABB1, ABB2, ABB3)
 - **Fin**
- **Opción 8:** Salir de la aplicación
 - Se termina la ejecución del programa
 - **Fin**

IA's Generativas

En esta práctica, solicité ayuda a una inteligencia artificial en tres ocasiones para complementar y mejorar mi código.

Primera Solicitud: Imprimir con comas

Inicialmente, compartí con la IA el siguiente fragmento de código, en el que se deseaba imprimir una lista de ISBN separados por comas, excluyendo la coma final:

```
def recorrer_inorder(self):  
  
    self.recorrer_inorder_recursivo(self.raiz)  
  
    if self.raiz is None:  
  
        print("El Árbol está vacío.")  
  
        return None  
  
def recorrer_inorder_recursivo(self, libro_actual):  
  
    if libro_actual:  
  
        self.recorrer_inorder_recursivo(libro_actual.izquierda)  
  
        print(f"{libro_actual.ISBN}", end = " ")  
  
        self.recorrer_inorder_recursivo(libro_actual.derecha)  
  
    return
```

La respuesta de la IA fue la siguiente, en la que realizo los cambios solicitados:

```
def recorrer_inorder(self):  
  
    if self.raiz is None:  
  
        print("El Árbol está vacío.")  
  
        return None  
  
    resultado = []  
  
    self.recorrer_inorder_recursivo(self.raiz, resultado)  
  
    print(", ".join(str(isbn) for isbn in resultado))  
  
def recorrer_inorder_recursivo(self, libro_actual, resultado):  
  
    if libro_actual:  
  
        self.recorrer_inorder_recursivo(libro_actual.izquierda, resultado)  
  
        resultado.append(libro_actual.ISBN)  
  
        self.recorrer_inorder_recursivo(libro_actual.derecha, resultado)  
  
    return
```

Puntos destacados:

- Se creó una lista llamada resultado para almacenar los ISBN durante el recorrido en lugar de imprimirlos directamente.
- Al finalizar, se usa join() para unir los elementos de la lista en una cadena con comas, logrando una salida correcta sin una coma final.

Segunda Solicitud: Obtener Nodos

Tuve problemas para almacenar los nodos una para elegir el libro aleatorio en la simulación. Por lo que ChatGPT me proporcionó el siguiente código.

```
def obtener_nodos(nodo):  
  
    return [] if nodo is None else [nodo] + obtener_nodos(nodo.izquierda) +  
    obtener_nodos(nodo.derecha)
```

Nota: el funcionamiento de la función esta explicada en el apartado código principal.

Tercera Solicitud: Uso del try catch

Le pregunte a al ChatGPT como se usa el try catch en Python a lo que esta me respondió con el siguiente código:

```
while True:  
  
    entrada = input("Introduce un número: ")  
  
    try:  
  
        numero = int(entrada)  
  
        break  
  
    except ValueError:  
  
        print("Eso no es un número válido. Intenta de nuevo.")
```

Puntos destacados:

- int(entrada) intenta convertirla a número.
- Si no puede, lanza una excepción ValueError, y el except lo atrapa.
- El bucle se repite hasta que la conversión sea exitosa.